Versión 0.5 — 3 de octubre de 2026

Unidades: 0.1 (un objeto o varios) y 0.2 (grupos por atributo)

Novedad de esta versión: cada respuesta dice las DOS cosas

Cuando escribes objetos con atributos, el sistema contesta **en dos capas**:

- **PLURALIDAD (0.1):** cuántas cosas hay — `es UN OBJETO` o `son VARIOS OBJETOS`.
- **ATRIBUTOS (0.2):** si comparten algún atributo, y cuáles.

Así ya no esconde lo que sabe: `casa(color=azul); carro` **no** dice un «indefinido» a secas, sino «son **varios objetos**, y en cuanto a atributos **no se puede decidir** (del color del carro no sabemos nada)».

Y hay una regla nueva, la de la **fusión**:

Escribes	Qué entiende
`casa(color=blanco); casa(forma=cuadrada)`	**La misma casa**, descrita dos veces: sus datos se suman → `casa blanca y cuadrada`, **un solo objeto**
`casa(color=blanco); casa(color=rojo)`	**Se contradicen** (mismo atributo, dos valores) → no elige: `no_decidible`
`casa, casa`	**Un solo objeto** (el nombre manda) — si no, sería un grupo y eso rompería 0.1

Novedad de esta versión: el vocabulario, arreglado

Carlos destapó una contradicción: `casa,carro` decía *grupo*, y `casa(color=blanco); carro(color=negro)` decía *no grupo*. La misma pareja de objetos, respuestas opuestas.

La causa era usar la palabra **grupo** con dos sentidos distintos. Ya está separado:

Unidad	La pregunta que responde	Respuestas
0.1	¿hay **una** cosa o **varias**? (sin contar)	`un_objeto` / `varios_objetos`
0.2	¿esos objetos **comparten** algo?	`grupo_por_atributo` / `sin_atributo_comun` / `una_sola_entidad`

La palabra GRUPO es solo de 0.2 y siempre quiere decir: varios objetos que comparten un atributo con el mismo valor. Así ya no se contradicen:

```
casa,carro                                -> varios_objetos
casa(color=blanco); carro(color=negro)    -> varios objetos, pero sin_atributo_comun
casa_A(color=blanco); casa_B(color=blanco)  -> grupo_por_atributo, comparten color
```

En una línea

Doble clic en `E:\Euclides\PROBAR.bat`. Se abre una ventana, corre tus casos y te deja escribir los tuyos. Para salir, escribe `salir`.

Qué sabe hacer hoy (y qué no)

0.1	Decidir si hay un objeto o varios objetos (sin contarlos)
0.2	**Reconocer** si varios objetos forman grupo y decir **qué atributos comparten**
0.2	**Formar** el grupo de los que cumplen un criterio (`color=rojo`)
Siempre	Contesta con **las dos capas**: cuántas cosas hay y si comparten atributos
Siempre	Si no entiende lo que le escribes, **lo dice**; nunca revienta
No sabe todavía	Contar. No dice "hay tres". No compara cantidades. No decide pertenencia

Las tres formas de preguntar

1. Un objeto o varios (unidad 0.1)

Cosas separadas por **comas**. Si repites un nombre, es **la misma** cosa señalada otra vez.

```
silla                -> un_objeto
silla,mesa           -> varios_objetos
silla,silla          -> un_objeto   (la misma silla, dos veces)
```

2. ¿Forman grupo? (unidad 0.2)

Objetos con atributos. Valen **punto y coma** o **comas de fuera** de los paréntesis. Los atributos van entre paréntesis como `nombre=valor`, separados por comas.

```
caja_A(color=rojo,forma=cubo); caja_B(color=rojo,forma=esfera)   -> grupo por color
caja_A(color=rojo,forma=cubo), caja_B(color=azul,forma=cubo)     -> grupo por forma
casa(color=blanco); carro(color=negro)                           -> sin_atributo_comun
```

3. Dame el grupo de... (unidad 0.2)

El criterio, dos puntos, y la escena.

```
color=rojo: a(color=rojo,forma=cubo); b(color=azul,forma=cubo); c(color=rojo,forma=esfera)
   -> el grupo son a y c
```

Lo más importante: el nombre es el identificador

El sistema **no ve** los objetos, y **no adivina** por el parecido de los nombres. El nombre que escribes **es** la identidad de la cosa.

- `silla` y `silla` → **la misma** silla.
- `silla_A` y `silla_B` → **dos** sillas distintas, aunque en el mundo las dos se llamen igual.

Y si escribes la misma cosa con dos descripciones que no cuadran:

```
casa(color=azul); casa(color=rojo)
   -> no se puede decidir: o es la misma casa (y una de las dos es falsa),
      o son dos casas y les falta un nombre distinto
```

Si escribes algo que no entiende

No revienta: te lo dice y te recuerda las formas correctas.

```
pregunta> caja(colorrojo)

NO ENTIENDO ESTO: no entiendo el atributo 'colorrojo'
   se escribe asi:  nombre=valor

FORMAS QUE SI ENTIENDO:
  0.1  un objeto o varios  silla,mesa
  0.2  forman grupo?       a(color=rojo,forma=cubo); b(color=azul,forma=cubo)
  0.2  dame el grupo de... color=rojo: a(color=rojo); b(color=azul)

El sistema no adivina: si no entiende, lo dice.
```

Las tres formas de lanzarlo

1. **Doble clic en** `PROBAR.bat` — lo normal. Corre tus casos y abre la ventana.
2. `py -3 E:\Euclides\probar.py "silla,mesa"` — una pregunta suelta.
3. `py -3 E:\Euclides\probar.py --casos` — todos tus casos de golpe.

Órdenes dentro de la ventana: `casos`, `ayuda`, `salir`.

Dónde escribes TUS casos

Archivo: `E:\Euclides\mis_casos.txt` (Bloc de notas). Cada línea:

```
pregunta | lo que espero | comentario
```

Lo que puedes poner en "lo que espero":

Escribes	Significa
`objeto` o uno	una sola entidad
`varios`	dos o más entidades distintas
`grupo:color`	forman grupo y comparten el color

Escribes	Significa
`grupo:color,forma`	comparten las dos cosas
`sin_atributo_comun`	son varios, pero no comparten nada
`una_sola_entidad`	solo hay una cosa (o es la misma descrita dos veces): la pregunta de grupo no aplica
`tiene:a,c`	al formar un grupo, el resultado son esos
`no_hay_grupo`	ninguno cumple el criterio
`no_decidible`	el sistema debe decir que no sabe
`no_entendido`	el sistema debe decir que no entiende la escritura
`?`	no comprobar nada, solo ver qué responde

Guarda el archivo y vuelve a lanzar el probador: **se recarga solo**. Los # son comentarios.

Por qué el sistema explica lo que dice

Cada respuesta sale con su **traza**: qué reglas se han probado, en qué orden, cuál ha decidido y qué conocimiento escrito la respalda. Ejemplo real:

```
PREGUNTA (0.2): forman GRUPO estos objetos?
ESCENA:
    casa  [color=blanco]
    carro [color=negro]
  reglas probadas, en orden:
    - reconocer_escena_vacia : escena_vacia(escena) -> falso
    - reconocer_entidad_contradictoria : hay_entidad_contradictoria(escena) -> falso
    - reconocer_una_sola_entidad : menos_de_dos_entidades(escena) -> falso
    - reconocer_atributo_indeterminado : algun_atributo_indeterminado(escena) -> falso
    - reconocer_grupo_por_atributo : hay_atributo_compartido_por_todos(escena) -> falso
    - reconocer_sin_atributo_comun : sin_atributo_compartido_por_todos(escena) -> cierto
RESULTADO: sin_atributo_comun
  lo decide la regla 'reconocer_sin_atributo_comun'
  respaldado por el conocimiento 'grupo_por_atributo' [FIJADO]
```

Qué buscar cuando lo pruebes

1. **Que 0.1 y 0.2 ya no se contradigan**: `casa,carro` contra `casa(color=blanco); carro(color=negro)`.
2. **La fusión**: `casa(color=blanco); casa(forma=cuadrada)` debería decir que es **la misma casa** con los datos sumados, no dos objetos.
3. **Cosas escritas a medias o mal**: `caja(colorrojo)`, `caja()`, `caja(color=)`. Todas deberían contestar "no entiendo", ninguna debería romperse.
4. **La misma cosa con dos descripciones distintas que sí chocan** (`casa(color=azul); casa(color=rojo)`).
5. **Un objeto al que le falta un atributo** que los demás sí tienen: debería callarse **en atributos** pero seguir diciendo que son varios objetos.

6. **Repetir el mismo nombre** en 0.1 (`taza, taza, taza`).
7. **Pedir un grupo que no existe** (`color=verde:` cuando nadie es verde).
8. Cosas tuyas: nombres con espacios, mayúsculas y minúsculas, números (`caja1, caja2`).

Si algo te chirría, dímelo con **la línea exacta que escribiste y lo que esperabas**.

Los archivos, por si quieres mirar

Archivo	Qué es
`E:\Euclides\conocimiento\*.kn`	El saber, en texto plano. Se puede leer y editar
`E:\Euclides\sistema\motor.py`	El programa que lee, decide y explica
`E:\Euclides\pruebas\`	Las pruebas del proyecto (0.1, 0.2 y robustez)
`E:\Euclides\mis_casos.txt`	**Tus casos**
`E:\Euclides\PROBAR.bat`	El lanzador de doble clic
`E:\Euclides\PROGRESO.md`	Qué está hecho, qué falla, qué toca
`E:\Euclides\diario\`	Lo que se hizo cada día y lo que se midió